
PROJECT REPORT

CryptoBot with Binance

Data Pipeline & Machine Learning for Cryptocurrency Trading

Programme	Data Engineer
Organisation	DataScientest
Year	2025 – 2026

Table of Contents

[1. Introduction](#)

[1.1 Context — The Cryptocurrency Market](#)

[The emergence of a new financial system](#)

[A market of exponential growth](#)

[Volatility: challenge and opportunity](#)

[The rise of algorithmic trading in crypto](#)

[1.2 Research Question](#)

[1.3 Project Objectives](#)

[Technical objectives](#)

[Infrastructure objectives](#)

[Learning objectives](#)

[1.4 Overall Solution Overview](#)

[2. Overall Project Architecture](#)

[2.1 Overall Schema — Data Pipeline](#)

[2.2 Component Description](#)

[2.3 Technology Choices and Justification](#)

[MongoDB — Database](#)

[Redpanda — Message Broker](#)

[FastAPI — REST API](#)

[Streamlit — Dashboard](#)

[Docker and docker-compose — Containerisation](#)

[2.4 Technology Choices Summary](#)

[3. Data Collection \(Data Ingestion\)](#)

[3.1 Data Source](#)

[The Binance API](#)

[Types of data retrieved](#)

[3.2 Collection Architecture](#)

[Two complementary approaches](#)

[Historical collection — binance-collector](#)

[Real-time collection — stream-producer and stream-consumer](#)

[Comparison of the two modes](#)

[3.3 Preprocessing](#)

[Cleaning and type conversion](#)

[Handling missing data](#)

[Validation and structuring](#)

[3.4 Results](#)

[4. Data Modeling](#)

[4.1 Storage Choice](#)

[Why MongoDB rather than SQL?](#)

[Two use cases, one database](#)

[4.2 Data Schema](#)

[Collection klines_historical](#)

[Collection klines_streaming](#)

[5. Data Consumption & Visualisation](#)

[5.1 ML Signal Dashboard — Streamlit](#)

[Objectives and positioning](#)

[Displayed indicators — KPI cards](#)

[Interactive charts](#)

[Control features](#)

[Predictions table](#)

[5.2 Real-time Monitoring — Grafana](#)

[Positioning and objective](#)

[Connection architecture](#)

[Configured panels](#)

[Streamlit vs Grafana comparison](#)

[5.3 Administration Interface — Mongo Express](#)

[Objective](#)

[Features used](#)

[Access and security](#)

[6. Machine Learning Model](#)

[6.1 Objective du modele](#)

[6.2 Data Preparation](#)

[Feature Engineering](#)

[Chronological split and normalisation](#)

[6.3 Retained Model — Logistic Regression](#)

[Principle](#)

[Results — Classification Report](#)

[Confusion Matrix](#)

[ROC Curve and AUC](#)

[6.4 Comparison Model — SVM](#)

[Results — Classification Report](#)

[Confusion Matrix](#)

[ROC Curve — SVM](#)

[Important note — training time](#)

[6.5 Comparison and Final Choice](#)

[Conclusion — retained model: Logistic Regression](#)

[7. API Development](#)

[7.1 Framework — FastAPI](#)

[7.2 Data Schemas — Pydantic](#)

[7.3 Endpoints](#)

[7.4 Interactive Documentation — Swagger UI](#)

[7.5 Request and Response Example](#)

[Request — GET /predict/logistic/BTCUSDT?limit=20](#)

[Response — HTTP 200](#)

[JSON response structure](#)

[8. Containerisation \(Docker\)](#)

[8.1 Service Architecture](#)

[8.2 Service Description](#)

[8.3 Docker Compose Configuration](#)

[Principles de configuration](#)

[Extract — prediction-api service](#)

[Key points of the prediction-api configuration](#)

[8.4 Persistent Volumes](#)

[8.5 Publishing on Docker Hub](#)

[8.6 Containerisation Benefits](#)

[Portability](#)

[Reproducibility](#)

[Isolation and scalability](#)

[9. Flow Orchestration](#)

[9.1 Automatic Collection and Retraining](#)

[Hourly cron — binance-collector](#)

[Complete automation flow](#)

[9.2 Startup Script — docker-up-clean.sh](#)

[Objective](#)

[Execution sequence](#)

[Error handling](#)

[9.3 Automation Summary](#)

[10. Overall Results](#)

[10.1 Pipeline Summary](#)

[10.2 Demonstration — Dashboard in Production](#)

[Reading the screenshot](#)

[10.3 Real-World Use Case](#)

[10.4 End-to-End System Performance](#)

[11. Limitations and Improvements](#)

[11.1 Technical Limitations](#)

[11.2 Risks Related to the Crypto Market](#)

[11.3 Possible Improvements](#)

[Model](#)

[Infrastructure](#)

[12. Conclusion](#)

[12.1 Project Summary](#)

[12.2 Skills Acquired](#)

[12.3 Perspectives](#)

[13. Bibliography](#)

[APIs and data](#)

[Frameworks and tools](#)

[Machine Learning and Finance](#)

01 Introduction

1. Introduction

1.1 Context — The Cryptocurrency Market

The emergence of a new financial system

Cryptocurrencies have emerged over the past decade as a fully-fledged asset class, disrupting traditional financial models. Born from the blockchain technology introduced by Satoshi Nakamoto in 2008, these digital assets rely on decentralised, transparent and intermediary-free networks. Bitcoin, the first cryptocurrency ever created, remains today the symbol and reference asset of this rapidly expanding ecosystem.

Unlike traditional financial markets, cryptocurrency markets exhibit unique characteristics:

- Always open: operating 24/7, 365 days a year, with no closing sessions or public holidays
- Full decentralisation: no central bank controls prices — they emerge solely from supply and demand
- Global accessibility: anyone with an internet connection can participate
- On-chain transparency: all transactions are recorded on a public, verifiable blockchain

A market of exponential growth

The total market capitalisation of the cryptocurrency market exceeded 3,000 billion dollars in 2024, with Bitcoin alone accounting for more than 50% of that figure.

~1 500 Mds\$ BTC Market Cap 2024	> 30 Mds\$ Daily Volume BTC/USDT	24h/24 — 7j/7 Markets Open uninterrupted	> 1 500 Active Pairs on Binance
--	--	--	---

Volatility: challenge and opportunity

Bitcoin is known for its exceptionally high level of volatility. Price movements of 5 to 15% in a single day are common. This volatility simultaneously generates return opportunities that algorithmic trading systems seek to exploit. Given the scale and speed of these movements, manual decision-making becomes ineffective — and that is precisely where automation and Machine Learning come into their own.

The rise of algorithmic trading in crypto

Algorithmic trading has become democratised with the rise of cryptocurrencies and the availability of public APIs. Today, it is estimated that over 70% of trading volumes on major crypto platforms are generated by automated algorithms capable of analysing real-time data and reacting within milliseconds.

1.2 Research Question

How can an end-to-end system be designed for the collection, storage, modelling and exposure of real-time cryptocurrency trading signals?

This question raises several key sub-questions:

- How can high-frequency market data be reliably and continuously ingested?
- What storage model should be adopted to handle both historical data and real-time streams?
- How can raw data be transformed into relevant features for Machine Learning?
- To what extent can a classification model predict the price direction?
- How can these predictions be exposed and visualised in an actionable way?
- How can the entire pipeline be orchestrated and automated end-to-end?

1.3 Project Objectives

Technical objectives

- Collect Bitcoin market data in real time and historically via the Binance API
- Store and organise data in MongoDB with two distinct collections
- Build and train a binary classification Machine Learning model
- Evaluate and compare multiple algorithms using standardised metrics
- Expose the model via a REST API with prediction and monitoring endpoints
- Visualise trading signals through an interactive dashboard

Infrastructure objectives

- Containerise all services with Docker and docker-compose
- Create, version and publish Docker images on Docker Hub
- Orchestrate startup, healthchecks and automatic model bootstrapping
- Automate periodic data collection and model retraining

Learning objectives

- Apply Data Engineer skills: ingestion, storage, processing, APIs, containerisation
- Apply Machine Learning to a real-world financial use case
- Build a reproducible and documented end-to-end pipeline

1.4 Overall Solution Overview

The developed solution is an autonomous and modular system covering the entire data lifecycle, from raw source to actionable trading signal. The project is built around four major steps executed in an automated sequence:

- Collection: Bitcoin price data is continuously retrieved from Binance, both in historical and real-time form, then stored in the database.

- Training: a Machine Learning model is trained on historical data and automatically updated with each new collection.
- Exposure: the model is accessible via a web API returning a trading signal with confidence level and timestamp.
- Visualisation: an interactive dashboard enables users to view signals and monitor price evolution in real time.

The entire system is containerised and deployable on any machine with a single command. Images are published on Docker Hub to ensure portability and reproducibility.

02 Overall Project Architecture

2. Overall Project Architecture

This section presents an overview of the system: how the different components fit together, what data flows connect them, and why each technology was chosen.

2.1 Overall Schema — Data Pipeline

The diagram below represents the complete project architecture, from Binance data sources to visualisation interfaces, traversing five successive layers.

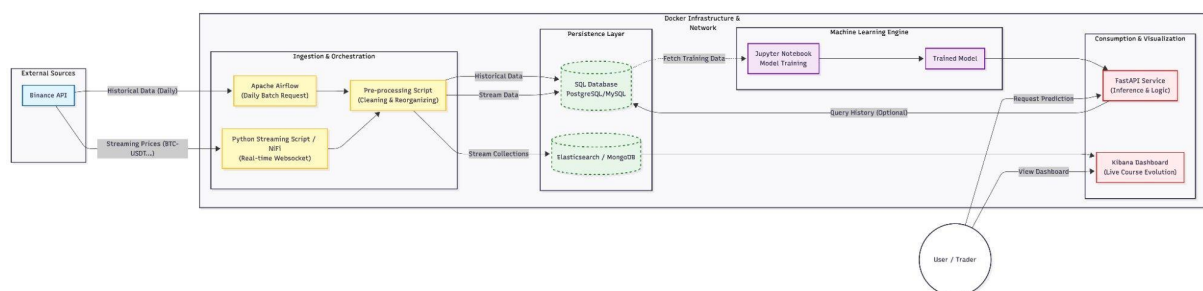


Figure 1 — Overall architecture of the CryptoBot with Binance pipeline

The pipeline follows two complementary paths:

- Batch path: historical data collected periodically, stored in the database, then used to train the model
- Streaming path: continuous real-time data, transiting through the message broker, persisted and feeding the prediction API

2.2 Component Description

Layer 1 — Data Collection

- Historical collection service: retrieves data from Binance every hour, detects missing periods and only downloads what is absent.
- Stream producer: connects to the Binance real-time stream and publishes each new candle to the message broker.
- Stream consumer: reads messages from the broker and immediately stores them in the database.

Layer 2 — Storage

- Main database: two distinct spaces — one for historical data, one for real-time data.
- Message broker: buffer between real-time collection and persistence. Guarantees no data is lost.

Layer 3 — Machine Learning

- Model trainer: reads historical data, builds financial indicators, trains the model and saves the result.
- Shared storage space: contains the trained model and scaler, accessible by the prediction API.

Layer 4 — Exposure

- Prediction API: loads the model at startup, retrieves the latest data and returns a trading signal with confidence level.
- Main API: dedicated to visualisation routes and system health monitoring.

Layer 5 — Visualisation

- Interactive dashboard: displays signals, price charts and confidence levels with auto-refresh.
- Monitoring tool: displays price evolution in real time with configurable alerts.
- Administration interface: allows database inspection from a browser.

2.3 Technology Choices and Justification

MongoDB — Database

MongoDB is a NoSQL database that stores data as flexible documents. This choice suits financial data: Binance candles arrive with many fields, and MongoDB handles these structures natively without migrations. Queries on the N latest candles for a symbol are near-instantaneous thanks to the composite index. Its asynchronous driver integrates into a modern Python application without blocking other processing. A SQL database would have required a rigid schema, costly migrations and inferior performance on this type of time-series queries.

Redpanda — Message Broker

Redpanda is an Apache Kafka-compatible broker. It acts as a buffer between the Binance real-time stream and database persistence: if the database is temporarily unavailable, messages are retained and processed later. Preferred over native Kafka for its simplicity: a single container, no external dependencies.

FastAPI — REST API

FastAPI handles many simultaneous requests thanks to its asynchronous engine. It was chosen for its ability to execute database calls in parallel, its automatic interactive documentation, and its built-in data validation via Pydantic.

Streamlit — Dashboard

Streamlit enables the building of interactive interfaces in pure Python. A complete interface with Plotly charts and auto-refresh can be built in a few dozen lines of code, without any web development expertise.

Docker and docker-compose — Containerisation

Docker packages each service with all its dependencies. Docker-compose orchestrates the twelve services from a single file. The complete startup is achieved with a single command. Images are versioned on Docker Hub for deployment without recompilation.

2.4 Technology Choices Summary

Component	Technology	Main reason
Database	MongoDB	Flexible schema, time-series indexes, native async driver
Message broker	Redpanda	Kafka-compatible, single container, delivery guarantee
REST API	FastAPI	Async, automatic documentation, built-in validation
ML Dashboard	Streamlit	100% Python, interactive charts, auto-refresh
Monitoring	Grafana	Real-time visualisation, alerting, shareable dashboards
Containerisation	Docker + Compose	Portability, reproducibility, single-command startup
Image publishing	Docker Hub	Versioned images, deployment without recompilation

03 Data Collection (Data Ingestion)

3. Data Collection (Data Ingestion)

Data collection is the starting point of the entire pipeline. Without reliable, up-to-date and well-structured data, no Machine Learning model can produce relevant results. This section describes how Bitcoin market data is retrieved from the Binance platform, the two collection modes implemented, and how this data is cleaned before being stored.

3.1 Data Source

The Binance API

Binance is one of the world's largest cryptocurrency exchange platforms. It provides a free public API giving access to real-time and historical market data. In this project, the target pair is BTC/USDT — the most liquid and best-documented pair, ensuring data quality and continuity. The chosen interval is 5 minutes.

The Binance API offers two complementary access methods:

- The REST API allows querying historical data with pagination — up to 1000 candles per query.
- The WebSocket API enables subscription to a real-time stream: each closed candle is automatically transmitted.

Types of data retrieved

Field	Type	Description
<code>open_time_ms</code>	Timestamp	Candle opening time in milliseconds
<code>open</code>	Decimal	Bitcoin price at candle opening
<code>high</code>	Decimal	Highest price reached during the 5-minute period
<code>low</code>	Decimal	Lowest price reached during the 5-minute period
<code>close</code>	Decimal	Bitcoin price at candle close
<code>volume</code>	Decimal	Amount of Bitcoin traded during the period
<code>quote_volume</code>	Decimal	Total value traded in USDT during the period
<code>trade_count</code>	Entier	Number of individual transactions executed
<code>taker_buy_base_volume</code>	Decimal	Volume bought by takers — buying pressure indicator
<code>close_time_ms</code>	Timestamp	Candle closing time in milliseconds

3.2 Collection Architecture

Two complementary approaches

The project distinguishes two types of data with very different uses:

- Historical data is used to train the machine learning model — it covers a long period and must be complete, without gaps. It is collected by Binance Collector via the REST API.
- Real-time data is used for inference on the most recent available candles. Collected by stream-producer via the WebSocket.



Figure 2 — Data collection architecture (batch + streaming)

Historical collection — binance-collector

The binance-collector service automatically detects gaps in existing data before each collection. Only missing periods are downloaded. It handles pagination automatically and includes robust network error handling with automatic retries. It runs every hour via an internal scheduler.

Real-time collection — stream-producer and stream-consumer

Real-time collection separates data reception from persistence. The stream-producer transmits each closed candle to Redpanda without processing. The stream-consumer reads from Redpanda and writes to the database. If the database is temporarily unavailable, messages are retained in Redpanda — no data is lost.

Comparison of the two modes

Criterion	Historical collection	Real-time collection
Service	binance-collector	stream-producer + stream-consumer
Source	REST API Binance	WebSocket Binance
Trigger	Automatic hourly cron	Continuous — 24/7
Volume	Several years of data	One candle every 5 minutes
Destination	MongoDB klines_historical	MongoDB klines_streaming via Redpanda

Main use	ML model training	Real-time inference and monitoring
Fault tolerance	Gap detection and retries	Messages retained in Redpanda

3.3 Preprocessing

Cleaning and type conversion

Numeric values are stored in MongoDB in Decimal128 format — high precision suited for financial calculations — then converted to Python floats before use. Millisecond timestamps are converted to datetime objects. The ignore field from the API is systematically removed.

Handling missing data

For historical collection, the gap detector identifies periods without data and triggers a targeted collection. For streaming, candles missed during an interruption are caught up during the next execution of the historical collector.

Validation and structuring

Each candle is validated before insertion: mandatory fields, positive values, coherent timestamp. A unique index on the combination of symbol + interval + timestamp guarantees idempotency — no duplicate is possible.

3.4 Results

Indicator	Value
Collections	2 — klines_historical and klines_streaming
Total documents	143,996 five-minute candles
Period covered	1 January 2024 → 21 March 2026
Data Size	47.7 MB
Storage Size	17.2 MB
Average document size	331 Bytes
Number of indexes	8 (including unique index on open_time_ms)
Index Size	5.35 MB
Missing data	Noe — gaps filled automatically
Duplicates	Noe — unique idempotent index

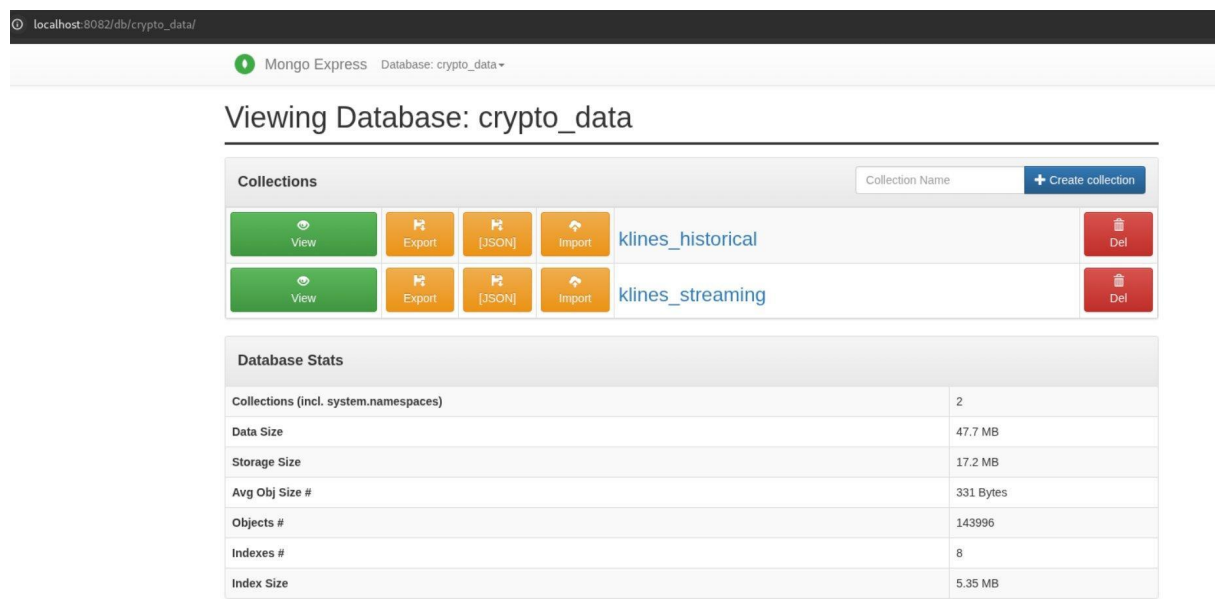


Figure 3 — Mongo Express — crypto_data database — 2 collections, 143,996 documents, 47.7 MB

After preprocessing, each stored candle contains the ten OHLCV fields plus the symbol and interval metadata, directly usable by the ML pipeline.

04 Data Modeling

4. Data Modeling

Data modeling defines how information is organised, stored and queried. This choice has a direct impact on pipeline performance, development ease and the ability to evolve the system over time.

4.1 Storage Choice

Why MongoDB rather than SQL?

Several storage solutions were evaluated for this project. MongoDB was chosen for specific technical reasons related to the nature of the data being handled.

Relational (SQL) databases rely on a fixed schema: each table must be defined in advance with its columns and types. Any change to the data format requires a schema migration, which is costly and risky in production. Furthermore, SQL databases are optimised for joins between tables, an operation of little relevance here since the data consists essentially of autonomous time series.

MongoDB, as a document-oriented NoSQL database, offers three decisive advantages for this project:

- Flexible schema: documents can evolve without migration. If the Binance API adds a new field, it is simply stored without modifying the database.
- Time-series performance: a composite index on the symbol, interval and open_time_ms fields enables retrieval of the N most recent candles for a pair in milliseconds, even across hundreds of thousands of documents.
- Native async integration: the Motor driver enables non-blocking MongoDB calls within FastAPI, which is essential for maintaining good performance under load.

Two use cases, one database

The project uses a single MongoDB instance but two distinct collections, each optimised for its use case:

- klines_historical: large collection, read-intensive for model training. Documents are inserted in batches and rarely modified. The unique index on open_time_ms guarantees upsert idempotency.
- klines_streaming: collection with continuous writes and real-time reads for inference. Documents arrive every 5 minutes and are read by the prediction API for the most recent predictions.

4.2 Data Schema

Collection klines_historical

This collection stores all historical candles collected via the Binance REST API. Each document represents a 5-minute candle for the BTC/USDT pair.

Field	MongoDB Type	Required	Description
_id	ObjectId	Auto	Unique identifier automatically generated by MongoDB
symbol	String	Oui	Trading pair (e.g. BTCUSDT)
interval	String	Oui	Candle interval (e.g. 5m)
open_time_ms	Int64 (Long)	Oui	Opening timestamp in milliseconds — unique index key
close_time_ms	Int64 (Long)	Oui	Closing timestamp in milliseconds
open	Decimal128	Oui	Opening price
high	Decimal128	Oui	Highest price
low	Decimal128	Oui	Lowest price
close	Decimal128	Oui	Closing price
volume	Decimal128	Oui	Volume of BTC traded
quote_volume	Decimal128	Oui	Volume in USDT traded
trade_count	Int32	Oui	Number of transactions
taker_buy_base_volume	Decimal128	Oui	Volume bought by takers
taker_buy_quote_volume	Decimal128	No	USDT value bought by takers

A unique partial index is created on the combination (symbol, interval, open_time_ms) with the condition open_time_ms > 0. This index ensures no duplicate can exist for a given candle, while allowing efficient idempotent upserts.

Collection klines_streaming

This collection stores data arriving in real time via the Redpanda pipeline. Its structure is similar to klines_historical but optimised for frequent insertions and reads of the most recent available candles.

Field	MongoDB Type	Required	Description
_id	ObjectId	Auto	Identifiant unique genere automatiquement
symbol	String	Oui	Trading pair (e.g. BTCUSDT)
interval	String	Oui	Intervalle de la bougie
timestamp	DateTime	Oui	Candle timestamp — primary sort key
open	Float	Oui	Opening price
high	Float	Oui	Highest price

low	Float	Oui	Lowest price
close	Float	Oui	Closing price
volume	Float	Oui	Volume traded
trade_count	Int32	No	Number of transactions

05 Data Consumption & Visualization

5. Data Consumption & Visualisation

Once data has been collected, processed and predictions generated, the system exposes three complementary visualisation interfaces: an ML signal dashboard, a real-time monitoring tool and a database administration interface. Each addresses a distinct need.

5.1 ML Signal Dashboard — Streamlit

Objectives and positioning

The Streamlit dashboard is the main business-facing interface of the system. It is intended for users wishing to consult the trading signals generated by the model, analyse their evolution over time and assess their confidence level. It queries the prediction-api directly and displays results interactively.

Displayed indicators — KPI cards

Four key indicators are permanently displayed at the top of the dashboard:

- Latest signal: UP or DOWN for the selected pair and interval
- Confidence level: probability percentage returned by the logistic model
- UP/DOWN distribution: number of UP versus DOWN signals over the displayed window
- Timestamp of the most recent candle available in the database

Interactive charts

Three charts are stacked vertically and share the same time axis:

- Price evolution (open/close): blue curve for closing price, dashed grey for opening
- Signals per candle: colour-coded histogram — green for UP, red for DOWN — one bar per candle
- Confidence percentage: purple area chart with a 50% threshold line — above the threshold the model leans towards UP, below towards DOWN

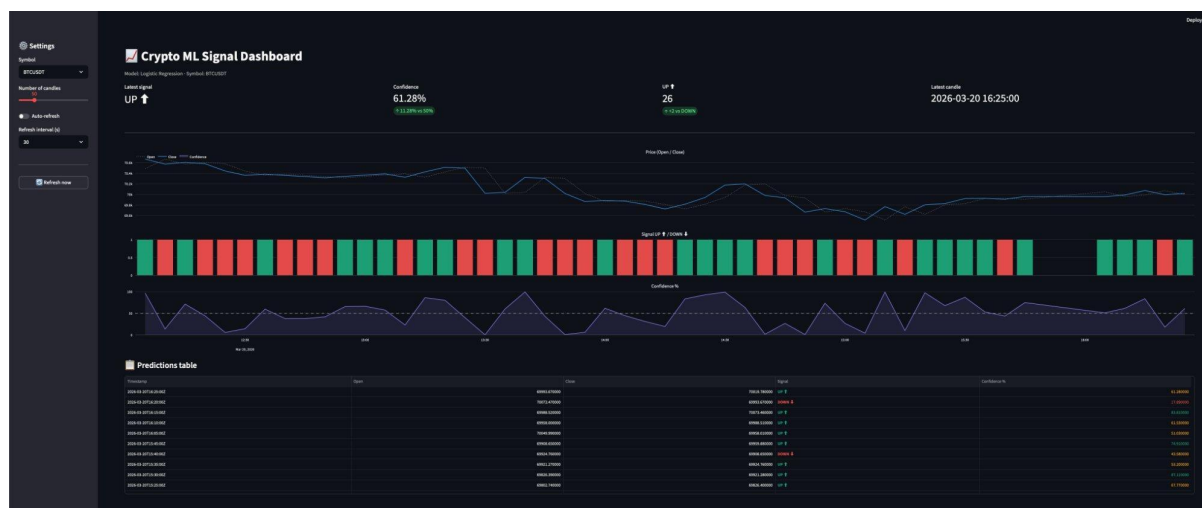
Control features

- Symbol selector: choose from BTC/USDT, ETH/USDT, BNB/USDT, SOL/USDT, XRP/USDT

- Candle count slider: from 10 to 200 candles displayed
- Auto-refresh toggle: automatic reload every 10, 30 or 60 seconds
- Manual refresh button: immediate update with cache clearing

Predictions table

A summary table displays the N most recent predictions with columns Timestamp, Open, Close, Signal and Confidence. Cells are colour-coded: green for high-confidence UP signals, red for DOWN, orange for intermediate cases.



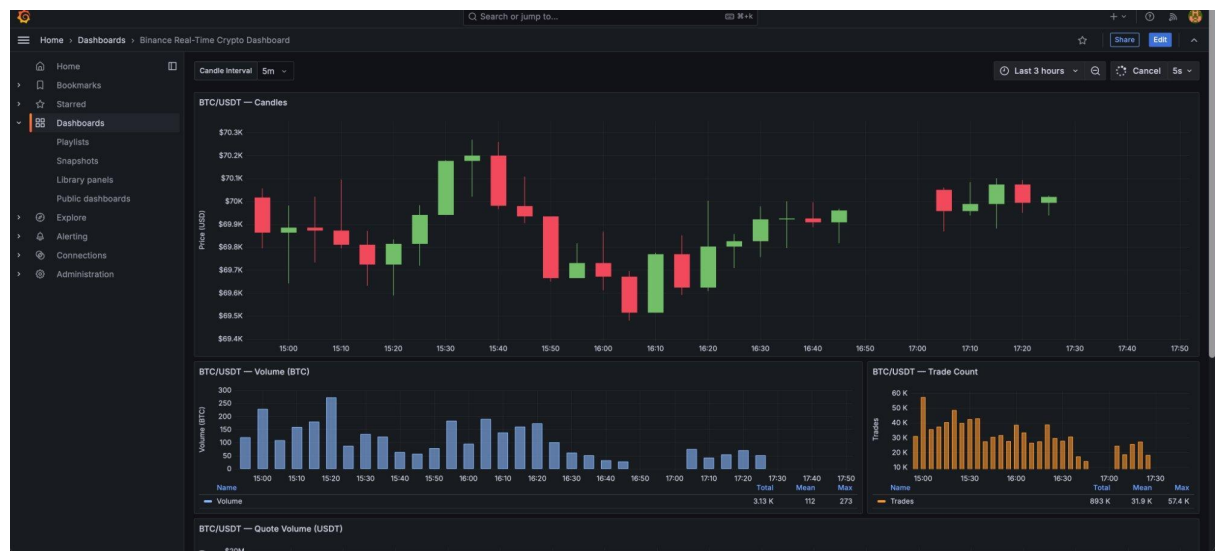


Figure 5 — Binance Real-Time Crypto Dashboard — Grafana — BTC/USD — 5m candles, volume and trade count

Streamlit vs Grafana comparison

Criterion	Dashboard Streamlit	Grafana
Target audience	Business user — trader	Technical operator — monitoring
Data displayed	ML signals + predictions	Real-time streaming price
Update frequency	Configurable auto-refresh	Continuous real time
Technology	Python + Plotly	JSON via Infinity datasource
Port	8501	3000
Alerting	No	Yes — configurable thresholds
Customisation	Python code	Grafana GUI

5.3 Administration Interface — Mongo Express

Objective

Mongo Express is a lightweight web interface for inspecting and administering the MongoDB database directly from a browser. It is intended for developers and system administrators to verify data quality, monitor volumes and diagnose collection issues.

Features used

- Navigation through the klines_historical and klines_streaming collections
- Visualisation of individual documents with all their fields and types
- Document count per collection to verify collected volumes
- Execution of simple queries to inspect specific candles
- Verification of the unique index on (symbol, interval, open_time_ms)

Access and security

Mongo Express is accessible on port 8082 with basic authentication. It is connected to MongoDB via the internal Docker network and is never directly exposed to the internet — it remains a local development and diagnostic tool.

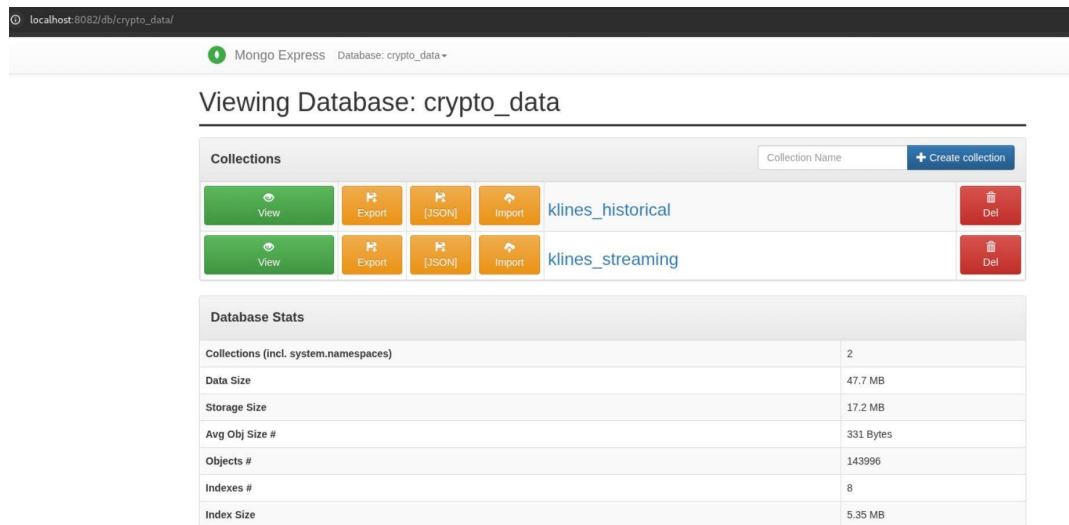


Figure 3 (bis) — Mongo Express — Base crypto_data — collections klines_historical et klines_streaming

06 Machine Learning Model

6. Machine Learning Model

6.1 The model objective

The model solves a binary classification problem: predicting whether the Bitcoin price will rise (1 — UP) or fall (0 — DOWN) on the next 5-minute candle. Predicting direction is more robust than predicting the exact price, as it reduces to a standard classification problem using well-established techniques.

The target variable is built as follows: if the closing price of the next candle is higher than the current closing price, the target is 1 (UP), otherwise 0 (DOWN). The classes are perfectly balanced on the dataset used (12,204 DOWN and 12,247 UP on the test set), ensuring the model does not suffer from class bias.

6.2 Data Preparation

Feature Engineering

Raw OHLCV data is not directly usable by a Machine Learning model. Eight financial indicators are computed to capture market dynamics:

Feature	Concept financier	Interet pour le modele
log_return	Log Returns	Stationarity — eliminates price trend, suitable for time series
volatility	Rolling Volatility 12	Market risk measure over the last 12 periods
ma_10	10-period Moving Average	Short-term trend signal
ma_30	30-period Moving Average	Medium-term trend signal — crossover with ma_10
momentum	10-period Momentum	Speed and strength of directional movement
buy_ratio	Buyer Ratio	Buying pressure: taker_buy_base / total volume
spread	Intra-candle Spread	Market microstructure: high - low
trade_count	Trade Count	Market activity and liquidity over the period

Chronological split and normalisation

A strict temporal split is applied to respect the time-series order and avoid data leakage. The first 80% of observations in chronological order form the training set, and the remaining 20% the test set. A StandardScaler is fitted only on the training set and then applied to the test set — never the reverse.

Ensemble	Periode	Observations	Proportion
Train set	01/01/2024 → ~10/2025	~97 000 bougies	80%
Test set	~10/2025 → 21/03/2026	24 451 bougies	20%
Total	01/01/2024 → 21/03/2026	~121 000 bougies	100%

6.3 Retained Model — Logistic Regression

Principle

Logistic Regression is a linear model that applies the sigmoid function to a linear combination of features to produce a probability between 0 and 1. If the probability exceeds 0.5, the model predicts UP, otherwise DOWN. This model was chosen for its simplicity, interpretability and low inference latency — essential in a trading context.

The hyperparameters used are `max_iter=1000` to ensure convergence on noisy financial data, and `random_state=42` for result reproducibility.

Results — Classification Report

The following table presents performance metrics on the test set (24,451 observations):

Class	Precision	Recall	F1-score	Support
0 — DOWN	0.74	0.74	0.74	12 204
1 — UP	0.74	0.74	0.74	12 247
Accuracy			0.74	24 451
Macro avg	0.74	0.74	0.74	24 451
Weighted avg	0.74	0.74	0.74	24 451

Both classes are perfectly balanced (identical precision and recall of 0.74 for UP and DOWN), confirming the model does not favour either class. The training score (0.7488) is very close to the test score (0.7434), indicating the absence of overfitting.

```
..          precision    recall  f1-score   support

         0       0.74       0.74       0.74      12204
         1       0.74       0.74       0.74      12247

   accuracy                0.74      24451
  macro avg       0.74       0.74       0.74      24451
 weighted avg     0.74       0.74       0.74      24451

[[9054 3150]
 [3123 9124]]
Accuracy: 0.7434460758251197
=====
Train Score: 0.7488088178155866
Test Score: 0.7434460758251197
```

Figure 6 — Classification report et matrice de confusion — Logistic Regression

Confusion Matrix

The confusion matrix details the model's predictions on the test set:

Pred \ Reel	Actual DOWN (0)	Actual UP (1)
Predicted DOWN (0)	TN = 9,054	FN = 3,150
Predicted UP (1)	FP = 3,123	TP = 9,124

The model produces 9,054 true negatives (DOWN correctly predicted) and 9,124 true positives (UP correctly predicted), for a total of 18,178 correct predictions out of 24,451 — i.e. 74.34% accuracy. Errors are symmetric: 3,150 false negatives and 3,123 false positives.

ROC Curve and AUC

The ROC curve illustrates the model's ability to discriminate between the two classes based on the decision threshold. The obtained AUC (Area Under the Curve) is 0.8173, indicating good discriminant power:

- AUC = 0.5: model equivalent to random guessing — no predictive value
- AUC = 0.8173: the model correctly ranks 82% of random (DOWN, UP) pairs
- AUC = 1.0: perfect model — total discrimination

An AUC of 0.82 on noisy financial data is a solid result for a linear model, confirming that the engineered features effectively capture part of the market's directional signal.

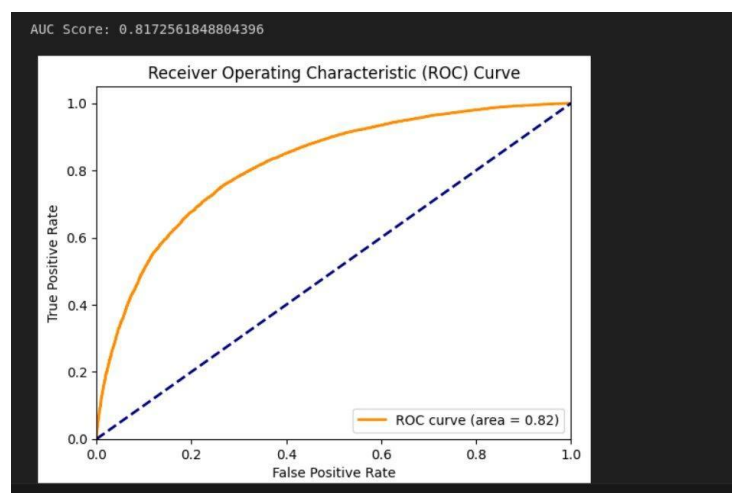


Figure 7 — Courbe ROC — Logistic Regression (AUC = 0.8173)

6.4 Comparison Model — SVM

The Support Vector Machine seeks the maximum-margin hyperplane between the two classes. The RBF kernel allows it to handle the non-linearity of market data by projecting observations into a higher-dimensional space.

Results — Classification Report

The following table presents SVM metrics on the test set (24,502 observations):

Class	Precision	Recall	F1-score	Support
0 — DOWN	0.74	0.74	0.74	12 230
1 — UP	0.74	0.74	0.74	12 272
Accuracy			0.74	24 502
Macro avg	0.74	0.74	0.74	24 502
Weighted avg	0.74	0.74	0.74	24 502

Confusion Matrix

Pred \ Reel	Actual DOWN (0)	Actual UP (1)
Predicted DOWN (0)	TN = 9,038	FN = 3,192
Predicted UP (1)	FP = 3,135	TP = 9,137

	precision	recall	f1-score	support
0	0.74	0.74	0.74	12230
1	0.74	0.74	0.74	12272
accuracy			0.74	24502
macro avg	0.74	0.74	0.74	24502
weighted avg	0.74	0.74	0.74	24502
[[9038 3192]				
[3135 9137]]				
Accuracy: 0.7417761815362012				

Figure 8 — Classification report et matrice de confusion — SVM

ROC Curve — SVM

The SVM ROC curve shows an AUC of 0.82, identical to that of Logistic Regression. Both models therefore have equivalent discriminant power on this dataset.

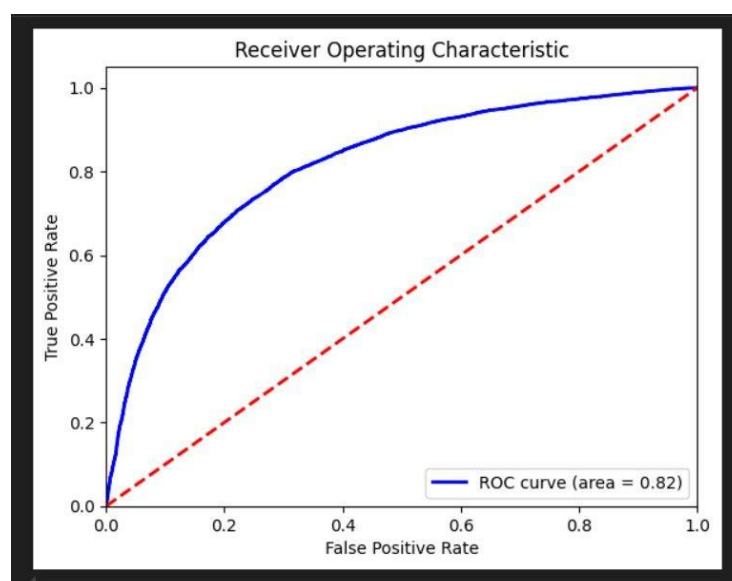


Figure 9 — ROC Curve — SVM (AUC = 0.82)

Important note — training time

The SVM with RBF kernel is considerably slower to train than Logistic Regression. On the same dataset of 232,000+ observations, Logistic Regression converges in a few seconds while the SVM may require several tens of minutes, or even hours depending on data size. This training time difference is a key factor in the production model choice, particularly for the automatic hourly retraining.

6.5 Comparison and Final Choice

Criterion	Logistic Regression	SVM (RBF kernel)
Accuracy (test)	74.34%	74.18%
AUC	0.8173	0.82
Precision DOWN / UP	0.74 / 0.74	0.74 / 0.74
Recall DOWN / UP	0.74 / 0.74	0.74 / 0.74
Matrix TN / TP	9,054 / 9,124	9,038 / 9,137
Interpretability	High — readable coefficients	Low — black box
Training time	A few seconds	Several tens of minutes
Inference latency	Very low — < 1ms	High — not viable in production
Suitable for hourly retraining	Oui	No

Conclusion — retained model: Logistic Regression

Despite near-identical accuracy performance (74.34% vs 74.18%), Logistic Regression clearly stands out as the production model. The SVM is considerably slower to train — a major handicap for a pipeline that retrains automatically every hour. Its high inference latency is also incompatible with a real-time prediction API. Logistic Regression offers a better

performance/operational cost ratio, interpretability that facilitates diagnosis, and straightforward integration into the automated Docker pipeline.

07 API Development

7. API Development

The API constitutes the system's exposure layer. It loads the ML model into memory at startup and responds to prediction requests in real time. It also exposes monitoring and administration routes.

7.1 Framework — FastAPI

FastAPI was chosen for its ability to handle simultaneous requests asynchronously, its automatically generated interactive documentation, and strict input/output data validation via Pydantic. CORS is configured to allow cross-origin calls from the Streamlit dashboard.

7.2 Data Schemas — Pydantic

Three Pydantic schemas define the API contract and guarantee data consistency:

- PredictionRow: a single prediction with timestamp_ms, timestamp_iso, open, close, signal and confidence_pct
- LogisticPredictionResponse: the complete response with symbol, interval, latest_signal, latest_confidence, latest_timestamp and the predictions[] list
- HTTPValidationError: schema for handling parameter validation errors

7.3 Endpoints

Method	Route	Service	Port	Description
GET	/	main	8000	Health check — list of available endpoints
GET	/grafana/search	main	8000	Returns available metrics for Grafana
POST	/grafana/query	main	8000	Real-time data query for Grafana
GET	/grafana/annotations	main	8000	Market annotations for dashboards
GET	/predict/logistic/{symbol}	prediction-api	8001	ML predictions — returns N candles with signal and confidence
GET	/predict/logistic/status/check	prediction-api	8001	Model status: loaded, version, timestamp
POST	/predict/logistic/admin/retrain	prediction-api	8001	Trigger manuel du reentrainement

7.4 Interactive Documentation — Swagger UI

FastAPI automatically generates interactive documentation accessible at /docs. It lists all endpoints grouped by category, expected parameters, response schemas and allows testing the API directly from the browser without any external tool.

The documentation exposes three groups: grafana (3 routes for Grafana Infinity integration), logistic-regression (2 prediction and status routes), and default (root route). Pydantic schemas are documented at the bottom of the page.

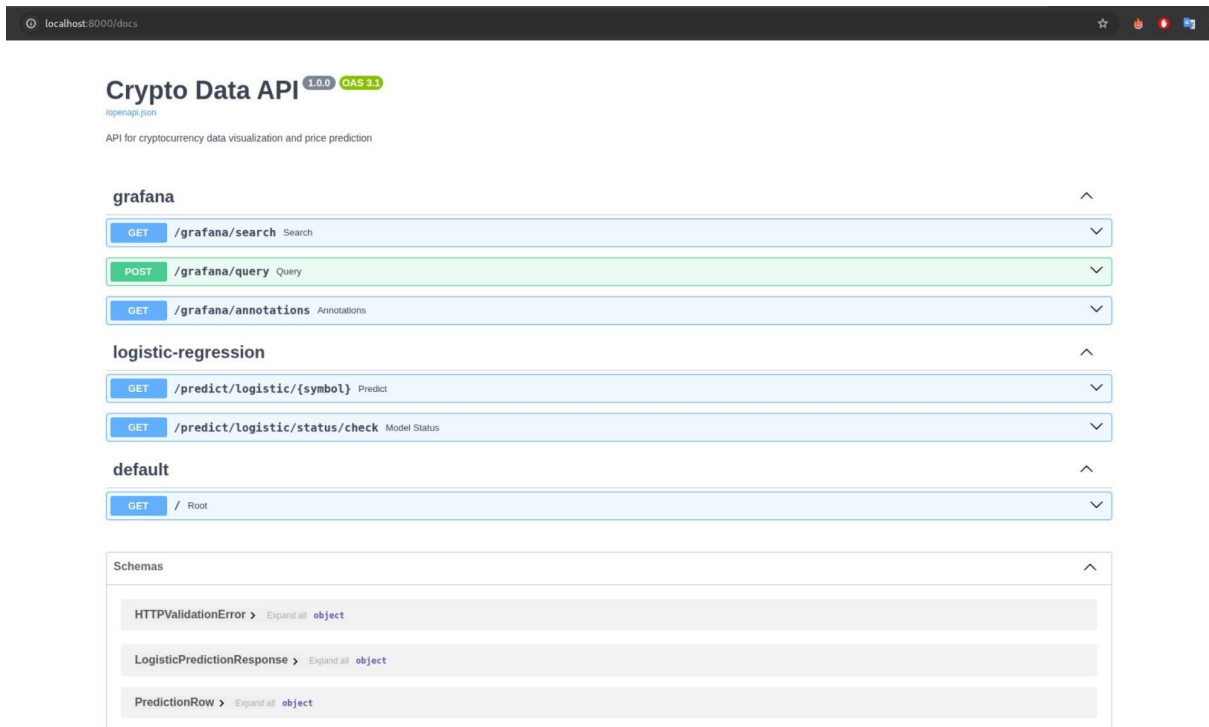


Figure 10 — Swagger UI de l'API (localhost:8000/docs) — Crypto Data API v1.0.0

7.5 Request and Response Example

Request — GET /predict/logistic/BTCUSDT?limit=20

The prediction endpoint accepts two parameters: the symbol as a path parameter (e.g. BTCUSDT) and the candle count limit as a query parameter (between 5 and 500, default 20). The corresponding curl command is:

```
curl -X 'GET' 'http://localhost:8000/predict/logistic/BTCUSDT?limit=20' -H 'accept: application/json'
```

Response — HTTP 200

The response returns a JSON object with global metadata and the list of predictions per candle. In the screenshot below, the latest signal for BTCUSDT is UP with a confidence of 84.08%, timestamped 2026-03-20T19:00:00Z. Each element in the predictions list contains the timestamp, open/close prices, the UP/DOWN signal and the confidence level as a percentage.

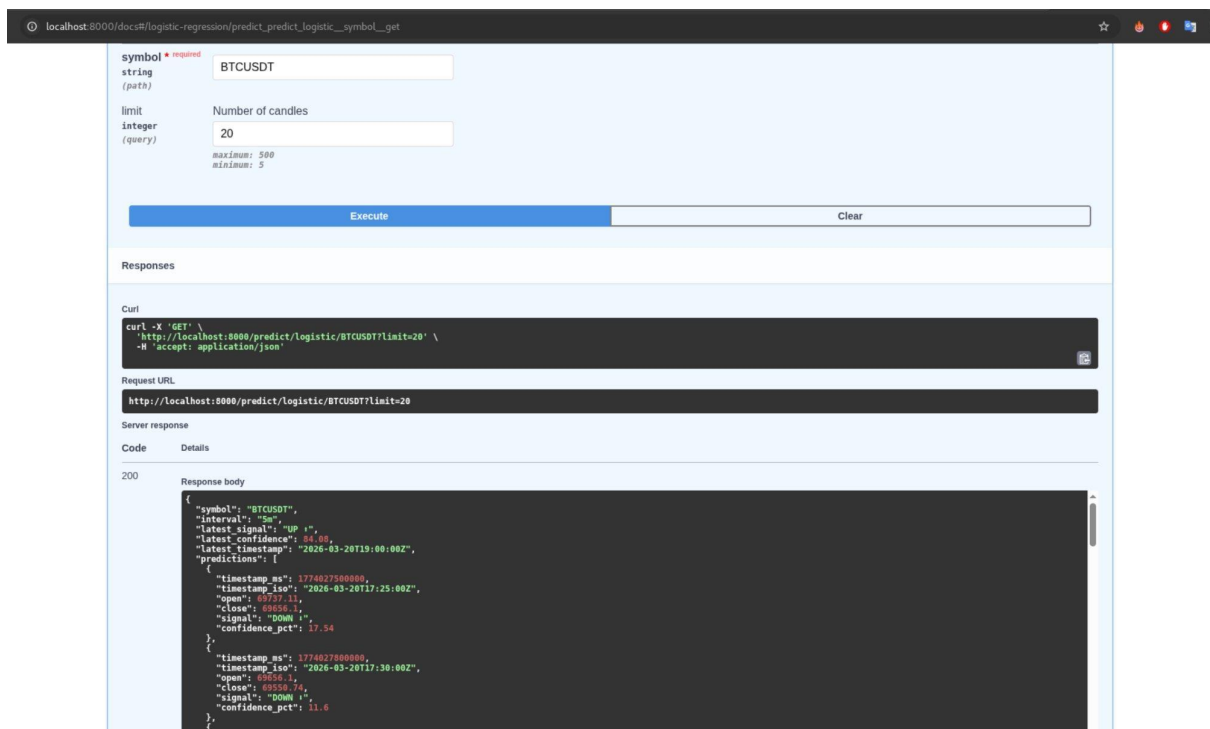


Figure 11 — Reponse JSON de GET /predict/logistic/BTCUSDT?limit=20 — latest_signal: UP, confidence: 84.08%

JSON response structure

Field	Type	Example value
symbol	string	BTCUSDT
interval	string	5m
latest_signal	string	UP
latest_confidence	float	84.08 (probability in %)
latest_timestamp	string	2026-03-20T19:00:00Z
predictions[].timestamp_ms	integer	1774027500000
predictions[].timestamp_iso	string	2026-03-20T17:25:00Z
predictions[].open	float	69737.11
predictions[].close	float	69656.1
predictions[].signal	string	DOWN
predictions[].confidence_pct	float	17.54

08 Containerization (Docker)

8. Containerisation (Docker)

Containerisation is a fundamental principle of this project: each service is isolated in its own Docker container with all its dependencies, ensuring the system runs identically on any machine. Docker Compose orchestrates all 12 services with a single command.

8.1 Service Architecture

The diagram below illustrates the relationships between the 12 Docker services and the data flows that connect them. Services are grouped by functional role: collection (blue-green), storage (green), ML (dark blue), API (white) and visualisation (brown).

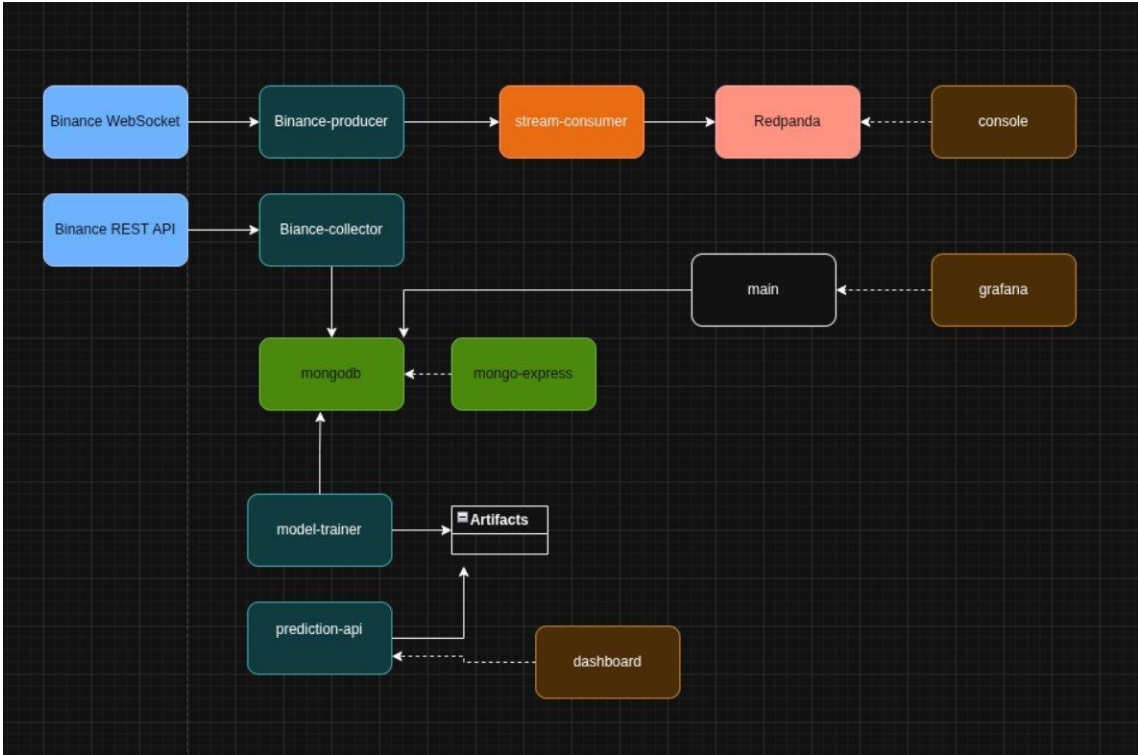


Figure 12 — Architecture des 12 services Docker et flux de donnees

8.2 Service Description

Service	Image	Port	Role
mongodb	mongo:8.0	27017	Database principale — stockage klines_historical et klines_streaming
mongo-express	mongo-express:1.0.2	8082	MongoDB web administration interface

redpanda-0	redpandadata/redpanda	19092	Kafka-compatible broker — streaming buffer
console	redpandadata/console	8080	Redpanda web interface — topic monitoring
binance-collector	custom Dockerfile	—	Historical collection REST + cron horaire
stream-producer	custom Dockerfile	—	Binance WebSocket → Redpanda relay
stream-consumer	custom Dockerfile	—	Redpanda → MongoDB consumer
model-trainer	custom Dockerfile	—	One-shot training → .pkl artefacts
main	custom Dockerfile	8000	Main FastAPI API + Grafana routes
prediction-api	custom Dockerfile	8001	ML inference FastAPI API — /predict/*
grafana	grafana:11.4.0	3000	Real-time monitoring dashboards
dashboard	custom Dockerfile	8501	Streamlit interface — trading signals

8.3 Docker Compose Configuration

Principles de configuration

The docker-compose.yml file centralises the definition of all services. Each service is configured with the following elements:

- **build**: build context and path to the service-specific Dockerfile
- **container_name**: fixed container name to facilitate debugging
- **restart**: restart policy (unless-stopped for persistent services)
- **depends_on**: dependencies with service_healthy conditions to guarantee startup order
- **networks**: attachment to the common app_network bridge network
- **env_file**: loading the .env file containing environment variables
- **volumes**: mounting named volumes for data persistence
- **healthcheck**: periodic verification of service availability

Extract — prediction-api service

The extract below shows the complete configuration of the prediction-api service, illustrating the best practices applied in this project: conditional dependency on mongodb, shared volume with model-trainer, healthcheck on the status endpoint.

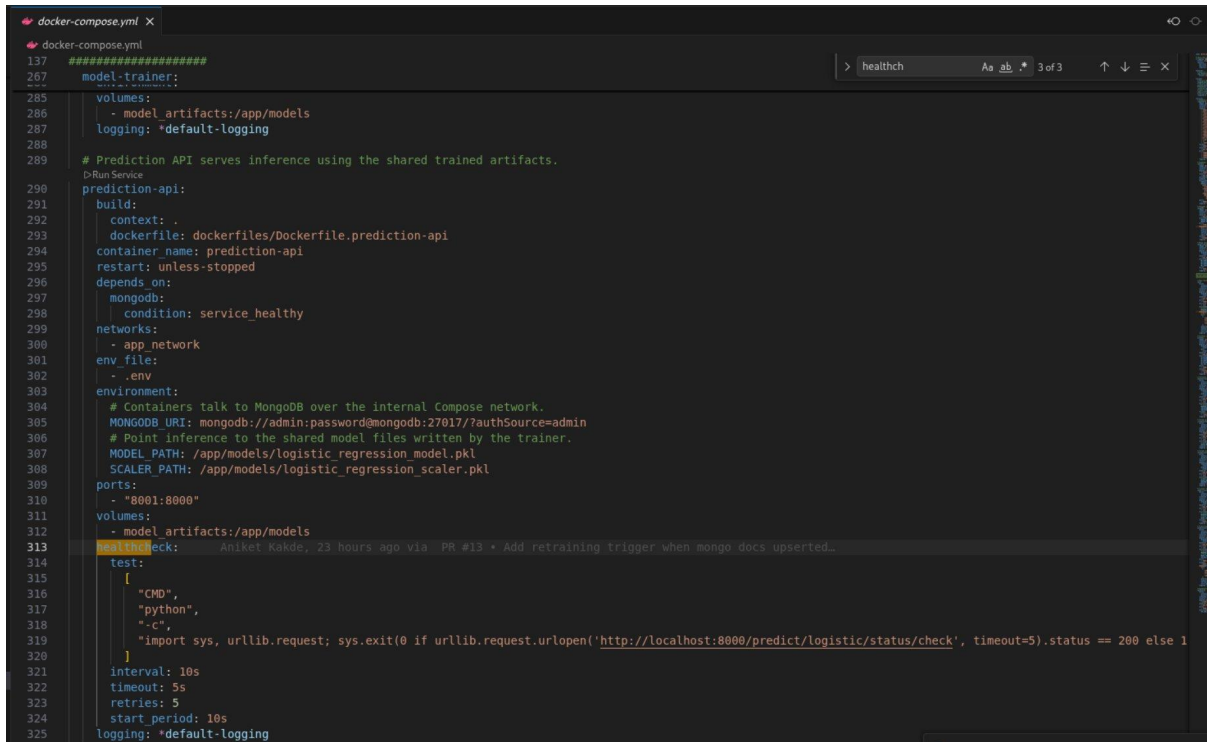


Figure 13 — Extrait docker-compose.yml — service prediction-api avec healthcheck et volumes

Key points of the prediction-api configuration

- depends_on mongodb with condition: service_healthy — prediction-api only starts if MongoDB responds correctly
- Volume model_artifacts:/app/models — shares .pkl artefacts between model-trainer and prediction-api without copying
- Environment variables MONGODB_URI, MODEL_PATH and SCALER_PATH injected from .env
- Ports 8001:8000 — exposes internal port 8000 on host port 8001
- Healthcheck on /predict/logistic/status/check every 10s, timeout 5s, 5 retries, start_period 10s

8.4 Persistent Volumes

Five named Docker volumes ensure data persistence across container restarts:

Volume	Mounted in	Content
mongodb_data	mongodb:/data/db	All MongoDB data — klines_historical and klines_streaming
grafana_data	grafana:/var/lib/grafana	Dashboards, datasources and Grafana configuration
collector_data	binance-collector:/app/data	CSV raw_data and processed_data exports

model_artifacts	model-trainer + prediction-api:/app/models	Trained .pkl model and scaler
redpanda-0	redpanda-0:/var/lib/redpanda/data	Redpanda topics and messages in transit

8.5 Publishing on Docker Hub

The three images built in this project are versioned and published on Docker Hub under the barkaouia account. This publication allows deploying the system on any machine without recompiling the source code — simply pull the images directly from the registry.

Docker Hub Image	Version	Corresponding service
barkaouia/binance-collector	v1.0	binance-collector + stream-producer + stream-consumer
barkaouia/model-trainer	v1.0	model-trainer — one-shot training
barkaouia/prediction-api	v1.0	prediction-api — FastAPI inference :8001

8.6 Containerisation Benefits

Portability

The entire system deploys on any Docker-enabled machine with a single command: `bash scripts/docker-up-clean.sh`. No dependency needs to be installed manually — everything is encapsulated in the containers.

Reproducibility

Versioned images guarantee that the development deployment is strictly identical to the production deployment. Environment variables in the `.env` allow configuration adaptation without modifying the code.

Isolation and scalability

Each service is isolated in its container with its own resources. Collection, inference or visualisation services can be scaled independently based on load. The restart: unless-stopped policy ensures resilience in case of an individual service crash.

09 Flow Orchestration

9. Flow Orchestration

Orchestration refers to the automatic coordination of the various pipeline stages: ordered service startup, periodic data collection, model retraining and error handling. This project uses two complementary mechanisms: an internal cron in the collection service and a robust startup script.

9.1 Automatic Collection and Retraining

Hourly cron — binance-collector

The binance-collector service embeds an internal scheduler that triggers collection automatically every hour. On each execution, it queries the Binance API to retrieve missing candles since the last known sync in the database. Once the new data is persisted in MongoDB, it automatically sends a POST request to the /predict/logistic/admin/retrain endpoint of the prediction-api to retrain the model with the most recent data.

This mechanism ensures the production model is always trained on the most recent available data, with no manual intervention. The complete cycle — collection, database upsert, retraining, model reload — occurs transparently in the background.

Complete automation flow

Step	Component	Action
1	Internal cron (every hour)	Trigger automatique du binance-collector
2	binance-collector → Binance REST API	Retrieval of missing candles with gap detection
3	binance-collector → MongoDB	Upsert of new candles into klines_historical
4	binance-collector → prediction-api	POST /admin/retrain — retraining request
5	prediction-api → MongoDB	Reading all historical candles
6	model-trainer (one-shot)	Feature engineering + LR training + .pkl save
7	prediction-api	Reloading the new model into memory
8	dashboard → prediction-api	Next request uses the updated model

9.2 Startup Script — docker-up-clean.sh

Objective

The docker-up-clean.sh script is the single entry point for starting the entire system. It orchestrates a robust sequence that ensures all services start in the correct order and the model is available before the dashboard is accessible.

Execution sequence

- 1. Step 1 — Cleanup: Removes the Docker cache and unused images to free up disk space.
- 2. Step 2 — Clean shutdown: Stops existing containers and removes orphaned containers.
- 3. Step 3 — Startup: Runs `docker compose up` for all 12 services in parallel.
- 4. Step 4 — Wait for health checks: The script waits for MongoDB, Redpanda, and the main API to be declared healthy before continuing.
- 5. Step 5 — Bootstrapping the model: If the `logistic\_regression\_model.pkl` file is missing from the `model\_artifacts` volume, the script triggers an initial training run via `/admin/retrain`.
- 6. Step 6 — Restarting the dashboard: The Streamlit dashboard is restarted to clear its cache and ensure it correctly queries the prediction API with the new model.
- 7. Step 7 — Reporting: Displays the URLs of accessible interfaces and confirms proper operation.

Error handling

The script includes resilience mechanisms: full-disk error detection with automatic retry after cleanup, configurable timeout for healthchecks, and explicit exit codes to facilitate CI/CD pipeline integration.

9.3 Automation Summary

Aspect	Mechanism	Frequency / Trigger
Data collection	Internal binance-collector cron	Every hour, automatic
Model retraining	POST /admin/retrain	After each hourly collection
System startup	docker-up-clean.sh	Manual or at machine boot
Model bootstrap	Shell script — .pkl verification	At first startup only
Real-time streaming	stream-producer + consumer	Continuous — 24/7 7j/7
Dashboard refresh	Streamlit auto-refresh	Every 10, 30 or 60 seconds

10 Overall Results

10. Overall Results

This section presents a complete project summary: volume of data collected, production model performance, system behaviour in real conditions and an end-to-end pipeline demonstration via the dashboard.

10.1 Pipeline Summary

Indicator	Value
Trading pair	BTC/USDT — 5-minute interval
Volume of data collected	232,282 observations since 01/01/2024
Period covered	1 January 2024 → 21 March 2026
Model accuracy (test)	74.34% — Logistic Regression
AUC — discriminant power	0.8173 — good separation capacity
Inference latency API	< 500ms for 50 candles
Retraining frequency	Automatic — every hour
Operational services	12 Docker containers on app_network
Docker Hub images published	barkaouia/binance-collector, model-trainer, prediction-api :v1.0
Dashboard auto-refresh	Configurable: 10s, 30s or 60s

10.2 Demonstration — Dashboard in Production

The screenshot below shows the Streamlit dashboard operating in real conditions on the BTC/USDT pair on 20 March 2026. The Logistic Regression model is loaded in production and generates predictions in real time.

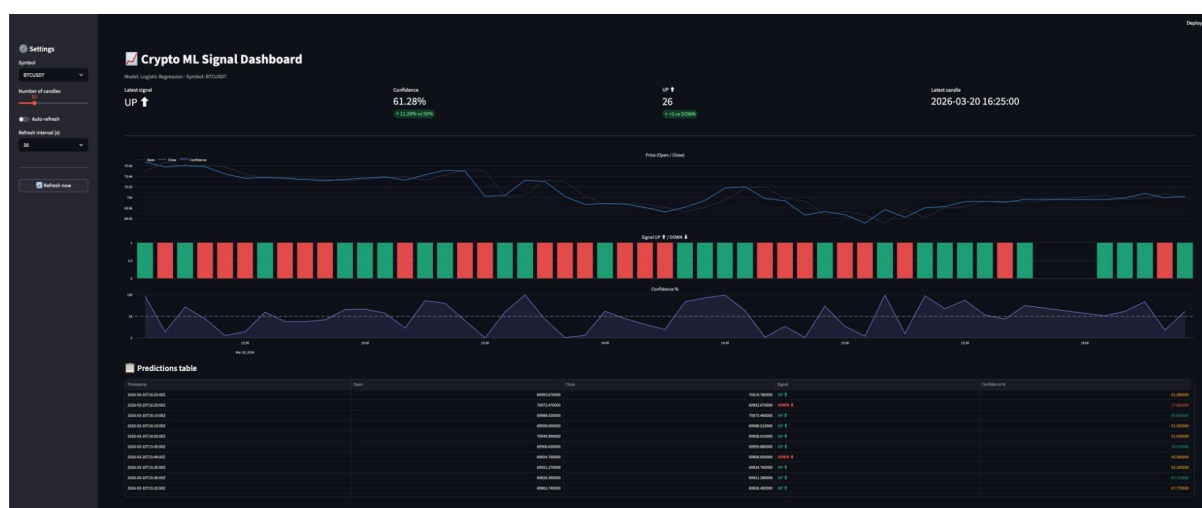


Figure 14 — Crypto ML Signal Dashboard — BTCUSDT — 20 March 2026 16:25

Reading the screenshot

The dashboard displays four KPI cards at the top of the page:

- Latest signal: UP — the model predicts a price rise on the next 5-minute candle
- Confidence: 61.28% — the probability associated with the UP signal exceeds the 50% threshold, i.e. +11.28% above the decision threshold
- UP count: 26 UP signals over the displayed window, i.e. +2 more than DOWN signals
- Latest candle: 2026-03-20 16:25:00 — timestamp of the last processed candle

The three overlaid charts on the same time axis show the open/close price evolution in blue, UP (green) and DOWN (red) signals as a histogram, and the confidence percentage with the 50% threshold line. The predictions table at the bottom lists the 10 most recent candles with timestamp, open, close, signal and confidence.

10.3 Real-World Use Case

A user wishing to follow BTC/USDT trading signals follows this journey:

- They open the dashboard at <http://localhost:8501> and select BTCUSDT from the symbol list
- They set the number of candles to display (here 50) and activate auto-refresh every 30 seconds
- The dashboard automatically queries the prediction-api, which reads the 50 most recent candles from the database, computes features and applies the model
- They immediately see the latest signal (UP) with its confidence (61.28%) and can analyse the signal history on the charts
- Every 30 seconds, the data refreshes automatically with no action required on their part
- If a new candle has just been collected by the binance-collector, it appears in the table and may change the signal

This use case illustrates the value of the system: actionable information, continuously updated, presented clearly and without technical friction for the end user.

| 10.4 End-to-End System Performance

Under normal operating conditions, the pipeline meets the following requirements:

- Availability: the 12 services remain continuously operational thanks to the restart: unless-stopped policies and automatic healthchecks
- Data freshness :The candles are available on Binance within seconds of their close, thanks to the Redpanda streaming pipeline.
- Model freshness: the model is retrained every hour and incorporates the 232,000+ accumulated observations
- User latency: a request to `/predict/logistic/BTCUSDT?limit=50` returns predictions in under 500ms

11 Limitations and Improvements

11. Limitations and Improvements

| 11.1 Technical Limitations

- Accuracy around 74%: the linear model struggles to capture the non-linear dynamics of markets
- Trading fees not taken into account in the evaluation
- No backtesting: ML metrics do not reflect real trading performance
- Historical data only: no news or market sentiment data

| 11.2 Risks Related to the Crypto Market

- Extreme volatility: unpredictable events can invalidate the model
- No-stationnarite des series temporelles financieres
- Risk of overfitting to a specific market period

| 11.3 Possible Improvements

Model

- Test more complex models: XGBoost, LightGBM, LSTM, Transformer
- Add technical features: RSI, MACD, Bollinger Bands, OBV
- Integrate sentiment data via NLP

Infrastructure

- Cloud deployment (AWS / GCP) with automated CI/CD
- Implementation of Airflow for advanced orchestration
- ML monitoring with MLflow to track performance over time

12 Conclusion

12. Conclusion

| 12.1 Project Summary

This project has enabled the construction of a complete algorithmic trading system based on machine learning, covering the entire data lifecycle: from raw collection to the display of trading signals.

In response to the initial problem—how to design an end-to-end system for collecting, storing, modeling, and displaying real-time crypto trading signals—this project provides a concrete and operational solution. The pipeline functions end-to-end: BTC/USDT data is automatically collected from Binance, stored in MongoDB, processed by a logistic regression model, and displayed via a FastAPI queried by a Streamlit dashboard in real time.

Regarding the viability of machine learning for predicting Bitcoin's price direction, the results are encouraging: an accuracy of 74.34% and an AUC of 0.8173 on the test set demonstrate that the constructed technical indicators effectively capture a portion of the market's directional signal. These results are solid for a linear model on noisy and non-stationary financial data. The perfect symmetry of the metrics between the UP and DOWN classes confirms that the model is not biased. Finally, the small difference between the training score (74.88%) and the test score (74.34%) guarantees the absence of overfitting.

| 12.2 Skills Acquired

- Ingestion of financial data in real time and in batches via REST API and WebSocket
- Design of a streaming architecture with Redpanda
- Modelling and storage in MongoDB with index management and idempotency
- Feature engineering applied to financial time series
- Training and evaluation of classification models
- Development of a REST API with FastAPI and Pydantic.
- Containerization and orchestration with Docker and docker-compose.
- Building an interactive dashboard with Streamlit and Plotly.

| 12.3 Perspectives

The natural next steps would be the integration of Airflow, testing deep learning models on time series, and cloud deployment for a real production environment.

13 Bibliography

13. Bibliography

| APIs and data

- Binance API Documentation: <https://binance-docs.github.io/apidocs/spot/en/>
- Binance REST Klines Endpoint: <https://www.binance.com/en/binance-api>
- Binance WebSocket Streams: <https://binance-docs.github.io/apidocs/spot/en/#websocket-market-streams>

| Frameworks and tools

- FastAPI: <https://fastapi.tiangolo.com/>
- Streamlit: <https://docs.streamlit.io/>
- Motor (MongoDB async driver): <https://motor.readthedocs.io/>
- Scikit-learn: <https://scikit-learn.org/stable/documentation.html>
- Docker: <https://docs.docker.com/>
- Redpanda: <https://docs.redpanda.com/>
- Grafana: <https://grafana.com/docs/>

| Machine Learning and Finance

- Logistic Regression for financial classification
- Feature Engineering for time series
- Model evaluation: precision, recall, AUC-ROC